

3. Firmware

This chapter documents in detail the ILS_NeoPixel_mega firmware, developed as a Custom Device for MobiFlight Community Firmware v11, which drives the entire ILS module (LOC/GS strips and DM13A backlighting) from an Arduino Mega 2560 Pro Mini.

3.1 Development Environment

Item	Detail
Target board	Arduino Mega 2560 Pro Mini (CH340G, COM4 port)
IDE / tooling	Visual Studio Code exclusively, with the PlatformIO extension
Terminal	PowerShell (Windows)
Framework	PlatformIO Core CLI, Arduino framework
LED library	FastLED
Base firmware	MobiFlight Community Firmware (Custom Device)

Table 16 — Firmware development environment.

Build command (mandatory before every flash):

```
$env:VERSION="1.0.1"; pio run -e ils_neopixel_mega_ils_neopixel_mega_mega
```

The VERSION environment variable absolutely must be set on the same line as the pio run command (semicolon separator in PowerShell, not && which isn't valid in this shell), otherwise the generated .hex file isn't named correctly and isn't recognized by MobiFlight when flashing.

3.2 Architectural History and Evolution

The firmware went through several significant architectural overhauls before reaching its final, stable version. This history is documented here because it explains certain design choices that might otherwise seem arbitrary.

3.2.1 Multi-Class Architecture Attempt (Abandoned)

An initial overhaul attempted to split the monolithic firmware into three independent MobiFlight Custom Devices (ILS_LOC, ILS_GS, LED_Tableau), each with its own .cpp/.h/.device.json files. This approach ran into a series of blocking issues:

- FastLED corruption caused by multiple calls to FastLED.addLeds() (one per class), which FastLED does not support.
- The Community firmware's MFCustomDevice.cpp file did not recognize the new device type names without modifying the firmware core.
- device.json files were silently ignored by MobiFlight in the absence of the "\$schema": "./mfdevice.schema.json" field — an omission that produces no visible error, making diagnosis time-consuming.
- Multiple calls to begin() corrupted FastLED's state.
- A board.json file must exist both in Community\ILS_NeoPixel_mega\boards\ AND in the root Boards\ folder — its absence from either location prevents compilation or board recognition.

3.2.2 Return to a Monolithic Architecture (Design Adopted)

Faced with these difficulties, the project reverted to a single-class architecture (ILS_NeoPixel_mega), driving both WS2812B strips and the DM13A backlighting from a single object, with one message identifier (messageID)

per function: 0 for LOC, 1 for GS, 2 for backlighting. This simpler architecture proved considerably more robust and forms the basis for all the later developments described in this chapter.

3.2.3 The LOC/GS Extinguishing Problem: A Multi-Stage Investigation

The most significant difficulty encountered on this project concerned the LOC/GS LED extinguishing logic when the ILS signal becomes invalid (out of range, tail-on to the runway, before signal capture). This problem required several rounds of diagnosis, each one revealing a deeper cause than the last. The table below summarizes this investigation, in chronological order.

#	Hypothesis / Design Tested	Result	Root Cause
1	Firmware timeout alone (3000 ms), no GS handling	GS never turned off	The update() code only checked the timeout for LOC — GS timeout handling was completely absent.
2	Added 2 MobiFlight outputs “ILS OFF LOC/GS” based on NAV HAS GLIDE SLOPE, in parallel with the main outputs based on ILS_ACTIVE	Random extinguishing during approach, LEDs sometimes stayed on when out of range	Race condition: two MobiFlight configs wrote to the same channel (same messageID) from two different, unsynchronized SimVars, with no execution-order arbitration.
3	Dedicated firmware gate ILS_ACTIVE (dedicated messageID), immediate extinguishing on receipt	Freezing reappeared: LEDs stayed on or off without updating	MobiFlight only sends data to the device when the computed final value changes (“send on change”) — if the LOC/GS index stays stable for several seconds (common during a stabilized approach), no more data is sent at all.
4	MobiFlight precondition (NAV HAS GLIDE SLOPE = 1/0) on the LOC/GS outputs themselves, with dedicated “OFF” outputs on the same variable	LEDs stayed off after a turn, despite a correct final value shown in the interface	Documented MobiFlight behavior: a config that goes from disabled (precondition false) back to enabled does not automatically push a new value to the hardware unless a new change is detected.
5	Final design: LOC/GS always active and continuously cached on the firmware side (no precondition), ILS_ACTIVE gate dedicated solely to display, immediate redisplay from cache on every toggle	Validated on a complete flight, no spurious extinguishing	—

Table 17 — History of successive diagnoses of the LOC/GS extinguishing logic.

3.3 Design final retenu

The final architecture rests on a simple principle: strictly separate receiving/caching data from displaying it.

- The LOC (messageID 0) and GS (messageID 1) messages are always transmitted by MobiFlight, with no precondition whatsoever, and are systematically cached by the firmware in two variables (`_lastLocIndex`, `_lastGsIndex`), whether or not they are being displayed at that moment.
- A dedicated ILS_ACTIVE message (messageID 3, `MSG_ILS_ACTIVE`) serves exclusively as a display “trigger”: at 0, both strips are turned off immediately; at 1, they are redisplayed immediately from the cached values, without waiting for a new LOC/GS message that might be delayed if the value hasn't changed in the meantime.
- The timeout mechanism, which had turned out to be the source of several extinguishing false positives (see Table 17, row 3), was removed entirely from the update() function. Extinguishing is now handled exclusively by this explicit gate.

This design removes the dependency on MobiFlight's “send on change” behavior for the extinguishing logic: a boolean that toggles (0→1 or 1→0) is always a value change, and is therefore always reliably transmitted — unlike a LOC/GS index rounded to 9 discrete levels, which can stay identical for several seconds in a row during a stabilized approach.

3.4 Header File Description (ILS_NeoPixel_mega.h)

The header file defines the hardware constants (pins, number of LEDs, brightness), the trigger message identifier, and the class declaration.

```
#define ILS_MAX_LEDS      9
#define ILS_BRIGHTNESS  80
#define ILS_PIN_LOC      7    // GPIO6 → D6  LOC  DIN
#define ILS_PIN_GS       6    // GPIO7 → D7  GS   DIN

#define DM13A_EN         5    // GPIO5  → D5  PWM
#define DM13A_LAT        49   // D49   libre
#define DM13A_SS         53   // GPIO53 → D53 SS

#define MSG_ILS_ACTIVE   3    // gate d’affichage
```

The class exposes the standard methods of a MobiFlight Custom Device (begin, attach, detach, set, update) plus three additional member variables specific to the final design: `_ilsActive` (gate state), `_lastLocIndex` and `_lastGsIndex` (cache of the last indexes received, initialized to -1 to represent “no data / out of range”).

3.5 Detailed Source File Description (ILS_NeoPixel_mega.cpp)

3.5.1 Color Palette

A static array of 9 RGB colors maps each LED index to a color, symmetric around the center (index 4, green): bright red at the extremes (maximum deviation), passing through dark orange, orange, and yellow.

3.5.2 The set() Function — Receiving MobiFlight Messages

This is the heart of the firmware, called by the MobiFlight connector on every message received over the serial link. The switch on `messageID` distinguishes five cases:

messageID	Function	Behavior
MSG_ILS_ACTIVE (3)	Display gate	Updates <code>_ilsActive</code> . If 0: both strips are turned off immediately. If 1: immediate redisplay from the cache (<code>_lastLocIndex</code> / <code>_lastGsIndex</code>) via <code>showIndex()</code> .
-1 / -2	Connector stop / Power Saving	Both strips turned off immediately (standard MobiFlight behavior).
0	LOC bar	Updates <code>_lastLocIndex</code> and its associated timestamp in every case; calls <code>showIndex()</code> only if <code>_ilsActive</code> is true.
1	GS bar	Same logic as LOC, on <code>_lastGsIndex</code> .
2	DM13A backlighting	Directly applies the brightness value (0-255) via <code>setBL()</code> , unrelated to the ILS_ACTIVE gate.

Table 18 — Behavior of the `set()` function by `messageID` (final version).

3.5.3 The showIndex() Function — Rendering on a Strip

A utility function shared by both strips: if the index received is outside the [0-8] range (which can happen transiently during large deviations, e.g. during a turn), the entire strip is turned off. Otherwise, only the LED matching the index is lit, with its palette color; all others are turned off.

```

void ILS_NeoPixel_mega::showIndex(CRGB* strip, int idx)
{
    if (idx < 0 || idx > 8) {
        fill_solid(strip, ILS_MAX_LEDS, CRGB::Black);
    } else {
        for (int i = 0; i < ILS_MAX_LEDS; i++) {
            strip[i] = (i == idx) ? PALETTE[i] : CRGB::Black;
        }
    }
    FastLED.show();
}

```

3.5.4 The update() Function — Called Every Loop

In the final version, this function is deliberately emptied of all extinguishing logic. It only keeps an initialization check, since extinguishing is now entirely the responsibility of the ILS_ACTIVE gate handled in set(). The _lastUpdateLOC / _lastUpdateGS variables are kept for potential future diagnostics but no longer trigger any action.

3.5.5 DM13A Functions (Backlighting)

Three functions manage the backlighting via the two chained DM13A chips: dm13a_byte() (sends one byte over bit-banged SPI), dm13a() (sends a full 32-bit mask to both chained chips, with the EN#/SS/LAT sequence), and setBL() (converts a 0-255 brightness value into a PWM signal on the EN# pin, knowing that a high value on EN# means off and a low value means full brightness — hence the 255-val inversion).



4. MobiFlight Configuration

This chapter documents the MobiFlight Community Firmware configuration needed to use the ILS_NeoPixel_mega firmware: Custom Device declaration (device.json), output configuration (SimVars, transforms, preconditions), and the internal ILS_ACTIVE variable.

4.1 Custom Device — device.json

The device.json file declares the device type to MobiFlight and the list of Message Types available in the interface (dropdown in the Display tab of each output). It must include the "\$schema" field to be recognized — its absence causes a silent failure (no error shown, the device simply doesn't appear).

```
{
  "$schema": "./mfdevice.schema.json",
  "Info": {
    "Label": "ILS NeoPixel Mega",
    "Type": "ILS_NeoPixel_mega",
    "Author": "francois.colas22",
    "Version": "1.0.3"
  },
  "Config": { "Pins": ["DIN LOC", "DIN GS"], "isI2C": false },
  "MessageTypes": [
    { "id": 0, "label": "LOC Index (0-8)" },
    { "id": 1, "label": "GS Index (0-8)" },
    { "id": 2, "label": "Luminosite DM13A (0-255)" },
    { "id": 3, "label": "ILS Active (0/1)" }
  ]
}
```

4.2 Output Configuration

The table below summarizes the final, flight-validated configuration of the four MobiFlight outputs required for the ILS module.

Output	Sim Variable	Transform (Modify)	Message Type	Precondition
ILS LOC	NAV CDI:1	Floor((\$+127)/254*8+0.5)	LOC Index (0-8)	none
ILS Glide	NAV GLIDE SLOPE:1	Floor(((\$*-1)+1)/2*8+0.5)	GS Index (0-8)	none
ILS ACTIVE OUT	NAV HAS GLIDE SLOPE:1	\$ (raw)	ILS Active (0/1)	none
Backlighting LED Panel	LIGHT POTENTIOMETER:3	Floor(\$/100*255)	Luminosite DM13A (0-255)	none

Table 19 — Final MobiFlight output configuration.

Points to watch, identified during the project:

- Transform functions are case-sensitive: Floor() with a capital F works; floor() in lowercase causes a silent parsing error in the MobiFlight logs ("Transform error... An error occurred on parsing your value formula"), with no visible sign anywhere else in the interface — only the application log reveals it.
- None of the LOC, Glide, or ACTIVE OUT outputs should carry a precondition in the final design: it's the firmware gate (ILS_ACTIVE) that handles extinguishing, not MobiFlight.
- LIGHT POTENTIOMETER:3 (not :21 or :85, two values tested by mistake during the project) is the SimVar that actually controls the panel lighting on the Baron 58.

4.3 Internal ILS_ACTIVE Variable

The ILS_ACTIVE gate is not directly an MSFS SimVar: it's an internal MobiFlight Variable, fed by NAV HAS GLIDE SLOPE:1. A MobiFlight Variable cannot be written from the Display tab of a standard output (“Output Device”): it requires the “Input Action” Display option, with action type “MobiFlight - Variable”.

Tab	Setting
Sim Variable	Source: Microsoft Flight Simulator, SimVar NAV HAS GLIDE SLOPE:1, Unit Bool
Modify	No transform (raw value)
Display	Choose = Input Action, trigger On Change, Action Type = MobiFlight - Variable, Type Number, Name ILS_ACTIVE, Value \$
Precondition	none

Table 20 — Configuration feeding the ILS_ACTIVE variable.

This variable is then used as a precondition (type “Config item”, directly referencing the “ILS_ACTIVE” config itself) for other specific needs on the MobiFlight side; the firmware, meanwhile, receives the signal state via the “ILS ACTIVE OUT” output from Table 19, which reads NAV HAS GLIDE SLOPE:1 directly and sends it to the physical device — the internal variable and the physical output share the same source but remain two separate configs.

4.4 Summary of SimVars Used

MSFS SimVar	Role in the Project	Range
NAV CDI:1	Localizer deviation (LOC)	-127 to +127
NAV GLIDE SLOPE:1	Glideslope error (GS)	-1.0 to +1.0
NAV HAS GLIDE SLOPE:1	Indicates whether the tuned station has a glideslope	0 or 1 (boolean)
LIGHT POTENTIOMETER:3	Baron 58 panel light rheostat	0 to 100

Table 21 — MSFS 2024 SimVars used by the ILS module.

Documented point to note: NAV HAS GLIDE SLOPE:1 indicates that the tuned frequency has a glideslope (station capability), not that the aircraft is currently receiving a valid, usable signal. After testing, no simple, reliable SimVar explicitly indicating “a valid ILS signal is being received right now” was identified in MSFS 2024 (NAV SIGNAL:1 and NAV CDI:1 were also tested for this specific purpose without success). The system accepts this residual limitation: in edge cases (frequency tuned but aircraft outside the useful geometry), a slight transient display mismatch remains possible, with no effect on normal operation, which has been thoroughly validated in flight.

4.5 Buzzer and Master Alarm / Master Caution Buttons

⚠ Warning — Unlike the ILS module (whose every config was built and verified step by step in MobiFlight, screenshots included), this part had not yet been detailed in this report. The content below comes from the project's summary notes and has not been re-verified config by config in the MobiFlight interface like the rest of this chapter — to be confirmed if the wiring or configuration has changed since.

The buzzer (NE555, §2.2.3) and the LEDs of the Master Alarm/Master Caution buttons (§2.4.6) are driven from MobiFlight via the GPIOs already identified in the Electronics chapter, triggered by the G1000's SimConnect events:

Output	GPIO	Trigger
Buzzer NE555 (BUZ1)	GPIO43	MASTER_WARNING_ACTIVE SimConnect event (continuous sound) or MASTER_CAUTION_ACTIVE (brief sound)
LED bouton Master Alarm (SW8)	GPIO37	Follows MASTER_WARNING_ACTIVE state
LED bouton Master Caution (SW7)	GPIO39	Follows MASTER_CAUTION_ACTIVE state

Table 22 — Buzzer and Master Alarm/Caution LED triggers (source: project summary notes, to be re-verified in MobiFlight).

The two buttons themselves (NC/NO contact, GPIO36 for SW8 and GPIO38 for SW7, cf. §2.4.6) are set up on the MobiFlight side as a Button-type Input Config, to allow the Master Alarm/Caution to be acknowledged (reset) from the physical panel.

i Info — To document this section with the same level of rigor as the ILS module (screenshots of the Sim Variable / Modify / Display / Precondition tabs), each relevant config would need to be reopened in MobiFlight — feel free to share them with me if you'd like me to fill in the table above with the exact transform details and the SimConnect event names used.



5. Troubleshooting — Lessons Learned

This chapter gathers, as a quick-reference table, all the symptoms encountered during the project, their cause, and the solution adopted. It is meant to speed up any future diagnosis on this module or a similar one.

Symptom	Cause	Solution
GS stays on indefinitely when out of range	Complete absence of timeout handling for GS in update() (initial bug)	GS timeout added, later replaced by the ILS_ACTIVE gate in the final design
Random extinguishing during approach	Race condition between two MobiFlight outputs writing to the same channel from different, unsynchronized sources	Unify all preconditions on a single source, then drop the precondition in favor of the firmware gate
LEDs freeze after a few seconds of stability	MobiFlight only sends a value to the device if it has changed; a rounded index can stay identical for several seconds	Dedicated boolean gate (always reliably transmitted since a 0/1 change is always detected) + firmware cache
LEDs off despite a correct “Final Value” shown in MobiFlight	Re-enabling a config disabled by a precondition doesn't automatically push a new value to the device	Remove all preconditions on the LOC/GS outputs; keep a precondition only on the gate itself, which is an always-transmitted boolean
“Transform error” in the logs, LEDs stuck at a fixed value	Lowercase floor() not recognized by MobiFlight's NCalc expression engine (case-sensitive)	Use Floor() with a capital F
device.json silently ignored by MobiFlight	Missing "\$schema" field in the file	Toujours inclure "\$schema": "./mfdevice.schema.json"
Incorrect .hex filename after compilation	VERSION variable not defined on the same line as pio run	Always use \$env:VERSION="1.0.1"; pio run -e ... (semicolon, not &&, in PowerShell)
“Choose variable” dropdown empty for a MobiFlight Variable-type precondition	This menu only lists variables already read by another output, not ones that are only written	Create an output that reads the variable and use it as a “Config item”-type precondition
LED lights up in an inconsistent color outside the useful range (turn, large deviation)	The raw value exceeds the range expected by the transform, producing an index outside 0-8	Accepted behavior: showIndex() automatically turns off if the index falls outside [0,8], which is the desired behavior in these transient cases
Master Caution LED (SW7) shifts to green when SW8 (Master Alarm) is activated at the same time	Likely coupling via a shared ground return between R4 (red, SW8) and R5 (red, SW7), not isolated from R10's return (green, SW7) — confirmed by a software test: forcing GPIO37 without physically pressing SW8 reproduces the color shift	To be fixed in the next PCB revision: ensure R4, R5, and R10 each independently reach a continuous ground plane, with no shared return path between them. A multimeter measurement (clean ground vs. R5/R10 junction) should be taken once the PCB is reopened
Master Caution LED (SW7) lit pure red instead of yellow	R- and G- on SW7 were wired in parallel on a single resistor (R5) on the common-anode side — the red LED (lower Vf) hogged the current, leaving the green channel nearly off	Resolved: added R10 (220Ω) dedicated to the green channel, in an independent series path from R5 (red channel) — each color now has its own current-limiting resistor, correct yellow blend obtained (SW7 alone, excluding

5 juillet 2026 9:00 PM

Symptom	Cause	Solution
		the interaction with SW8 — see next row)

Table 23 — Troubleshooting reference.



6. Conclusion

The ILS module (Localizer/Glideslope LED bar) of the G1000 Baron 58 cockpit is now functional and validated under real flight conditions: interception, capture, extended stabilized approach, and touchdown all proceed without spurious extinguishing or erratic behavior. The physical wiring is considered final.

The main lesson from this project goes beyond the strict scope of electronics or code: the central difficulty was never in the business logic itself (displaying a deviation index), but in the subtleties of MobiFlight's behavior as a middle layer — in particular its practice of only sending data on value change, and the incomplete refresh of configs when a precondition is re-enabled. These two behaviors, not obviously documented, led to several rounds of diagnosis before the root cause was clearly identified each time.

The final solution adopted — a strict separation between receiving/caching data (always active, never subject to a precondition) and displaying it (driven by a dedicated boolean gate, inherently reliable since a binary state change is always transmitted) — forms a robust, reusable architecture for any future module of this kind on the same platform.

Testing is currently continuing on other aircraft (Diamond DA62, DA40, Cessna 172 family) to confirm this configuration's portability to other Garmin avionics in MSFS 2024.

